

Program Representations

Java Bytecode and ASTs

Agenda

- Java Bytecode Part 2
- Program Representation 2: ASTs

Java Bytecode Part 2

Java Bytecode

- low-level, platform-independent set of instructions that the Java Virtual Machine (JVM) executes
- **Intermediate Representation**
- How is it different from the source?
 - Optimized (constant folding)
 - Not human readable

Example - What does this code do?

```
0: iload 1
1: iload 2
2: iadd
3: istore 3
4: iload 3
5: iconst_2
6: imul
7: ireturn
```

Local Variable Table

this	5	5	
------	---	---	--

Operand Stack

Warmup: Lec3 Worksheet Exercise 2

javap command

- Tool provided with the JDK (Java Developer Kit)
- Runs on the class file (binary) to give you the information that is stored there in human readable format

```
javap -c ClassName
```

Exercise 3

- Download Mystery.class from today's reading
- Run javap to inspect the bytecode
- **Exercise:** Work through it to figure out:
 - What does this code do?
 - What do you think the source looked like?

Method Calls

```
public static void main(String[] args) {
```

```
    int a = 1;
```

```
    int b = 2;
```

```
    int c = calc(a, b);
```

```
}
```

```
static int calc(int a, int b) {
```

```
    return (int) Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
```

```
}
```

```
public static void main(java.lang.String[]);
```

```
    0: iconst_1
```

```
    1: istore_1
```

```
    2: iconst_2
```

```
    3: istore_2
```

```
    4: iload_1
```

```
    5: iload_2
```

```
    6: invokestatic #2    // calc:(II)I
```

```
    9: istore_3
```

```
   10: return
```

```
static int calc(int, int);
```

```
    0: iload_0
```

```
    1: i2d
```

```
    2: ldc2_w           #3    // double 2.0d
```

```
    5: invokestatic    #5    // java/lang/Math.pow: (DD)D
```

```
    8: iload_1
```

```
    9: i2d
```

```
   10: ldc2_w           #3    // double 2.0d
```

```
   13: invokestatic    #5    // java/lang/Math.pow: (DD)D
```

```
   16: dadd
```

```
   17: invokestatic    #6    // java/lang/Math.sqrt: (D)D
```

```
   20: d2i
```

```
   21: ireturn
```

Constant Pool

Table that stores various constants needed for execution

Includes:

- Literals
- Symbolic references: accessing an object in memory using its name (or identifier) rather than its direct memory address

`invokestatic #2` executes the static method at the second index of the constant pool

Number sign refers to the constant pool

Method Calls

```
public static void main(String[] args) {
```

```
    int a = 1;
```

```
    int b = 2;
```

```
    int c = calc(a, b);
```

```
}
```

```
static int calc(int a, int b) {
```

```
    return (int) Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2));
```

```
}
```

```
public static void main(java.lang.String[]);
```

```
    0: iconst_1
```

```
    1: istore_1
```

```
    2: iconst_2
```

```
    3: istore_2
```

```
    4: iload_1
```

```
    5: iload_2
```

```
    6: invokestatic #2    // calc:(II)I
```

```
    9: istore_3
```

```
   10: return
```

```
static int calc(int, int);
```

```
    0: iload_0
```

```
    1: i2d
```

```
    2: ldc2_w           #3      // double 2.0d
```

```
    5: invokestatic    #5      // java/lang/Math.pow: (DD)D
```

```
    8: iload_1
```

```
    9: i2d
```

```
   10: ldc2_w           #3      // double 2.0d
```

```
   13: invokestatic    #5      // java/lang/Math.pow: (DD)D
```

```
   16: dadd
```

```
   17: invokestatic    #6      // java/lang/Math.sqrt: (D)D
```

```
   20: d2i
```

```
   21: ireturn
```

Non-static method calls

Method calls:

`invokevirtual` instead of `invokestatic`

Constructor calls (object creations):

`invokespecial`

Method Calls - nonstatic

```
class Example {  
    public void greet() {  
        System.out.println("hello");  
    }  
}  
  
Example e = new Example();  
example.greet();
```

```
0: new          #2          // Create a new Example object  
3: dup          // Duplicate the reference to the object on the operand stack  
4: invokespecial #3          // Call constructor (Example.<init>)  
7: aload_1      // Load the reference to the Example object (example)  
8: invokevirtual #4          // Call greet() on the instance of Example  
11: return
```

Given this source, what is the bytecode?

```
public class CountdownTimer {  
    public static void main(String[] args) {  
        CountdownTimer timer = new CountdownTimer();  
        timer.startCountdown(10);  
    }  
    public void startCountdown(int start) {  
        int count = start;  
        while (count > 0) {  
            System.out.println(count);  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {  
                System.err.println("Timer interrupted.");  
            }  
            count--;  
        }  
        System.out.println("Blast off!");  
    }  
}
```

```
public void startCountdown(int);  
  
Code:  
0: iload_1  
1: istore_2  
2: iload_2  
3: ifle 22  
  
6: getstatic #5 // Field java/lang/System.out:Ljava/io/PrintStream;  
9: iload_2  
10: invokevirtual #6 // Method java/io/PrintStream.println:(I)V  
13: invokestatic #7 // Method java/lang/Thread.sleep:(J)V  
16: iinc 2, -1  
19: goto 2  
  
22: getstatic #5 // Field java/lang/System.out:Ljava/io/PrintStream;  
  
25: ldc #8 // String Blast off!  
27: invokevirtual #6 // Method java/io/PrintStream.println:(Ljava/lang/String;)V  
30: return
```

Given this source, what is the bytecode?

```
public class CountdownTimer {  
    public static void main(String[] args) {  
        CountdownTimer timer = new CountdownTimer();  
        timer.startCountdown(10);  
    }  
    public void startCountdown(int start) {  
        int count = start;  
        while (count > 0) {  
            System.out.println(count);  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {  
                System.err.println("Timer interrupted.");  
            }  
            count--;  
        }  
        System.out.println("Blast off!");  
    }  
}
```

```
public CountdownTimer();  
Code:  
0: aload_0  
1: invokespecial #1 // Method java/lang/Object."<init>":()V  
4: return  
  
public static void main(java.lang.String[]);  
Code:  
0: new #2 // class CountdownTimer  
3: dup  
4: invokespecial #3 // Method "<init>":()V  
7: astore_1  
8: aload_1  
9: bipush 10  
11: invokevirtual #4 // Method startCountdown:(I)V  
14: return
```

Method Descriptors

- Specify the method's return type and parameter types
 - At a lower level than java source code
 - Notably does not include the method name
-
- `()I` – Describes a method that takes no parameters and returns an `int`.

Method Descriptors

1. Basic Types:

- `B` = `byte`
- `C` = `char`
- `D` = `double`
- `F` = `float`
- `I` = `int`
- `J` = `long`
- `L<classname>;` = an object of a class (e.g., `Ljava/lang/String;` for `String`)
- `S` = `short`
- `Z` = `boolean`

2. Void Return Type:

- `V` is used for methods that return `void`.

3. Array Types:

- For arrays, you use `[]` followed by the descriptor for the element type (e.g., `[I` for an array of `int`).

Examples:

Method with one **String** parameter and **void** return type:

- `(Ljava/lang/String;)V` – A method that takes a **String** and returns **void**.

Method with two parameters: **int** and **double**, and returns **boolean**:

- `(ID)Z` – A method that takes an **int** and a **double**, and returns a **boolean**.

Method with an array of **int** and a **String** parameter, returning **void**:

- `([ILjava/lang/String;)V` – A method that takes an array of **int** and a **String**, and returns **void**.

Exercise 4

- Download Mystery2.class from today's reading
- Run javap to inspect the bytecode
- **Exercise:** Work through it to figure out:
 - What does this code do?
 - What do you think the source looked like?

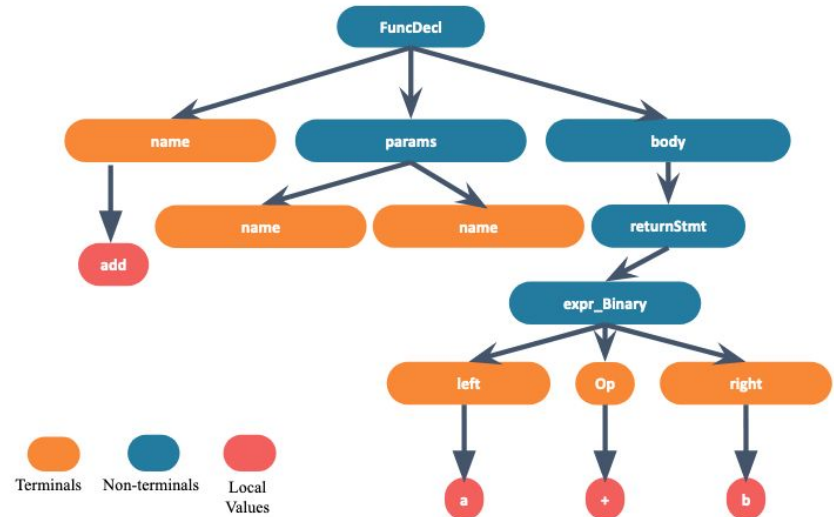
Program Representation 2: Abstract Syntax Trees

Representing Code as a Tree

Code has a natural hierarchical structure which cannot be captured as plain text

Classes have methods, methods have statements...

```
function add(a, b) { return a + b; }
```



AST Explorer

<https://astexplorer.net/> Make sure to switch the language to Java!

Answer the following questions:

1. What is the parent of the `name` identifier in the parameter of `addName`?
2. What is the parent of `methodDeclaration` for `addName`?
3. What is the parent of the `fieldDeclaration` for `addName`?
4. Change the field type from `ArrayList<String>` to `List<String>`. What changes?
5. Add a second parameter to `addName`. Where does it appear?

Java - Statements **vs** Expressions

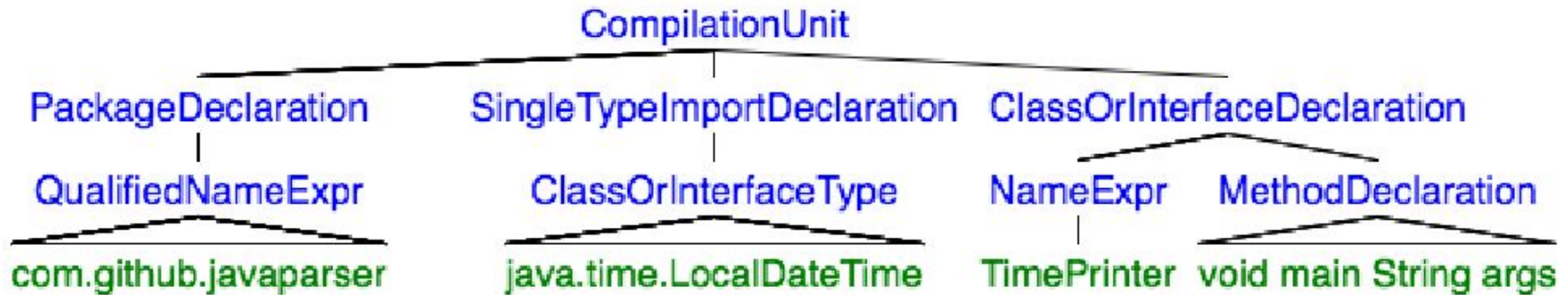
- A statement is a complete unit of execution that performs an action. It often ends with a semicolon
- An expression is a combination of variables operators, literals, and method calls that evaluates to a single value

Java - Statements **vs** Expressions

- `int x = 5;`
 - Statement
- `2 + 3`
 - Expr
- `System.out.println("Hello World");`
 - Statement
- `Math.sqrt(9)`
 - Expr
 - OR expression-statement

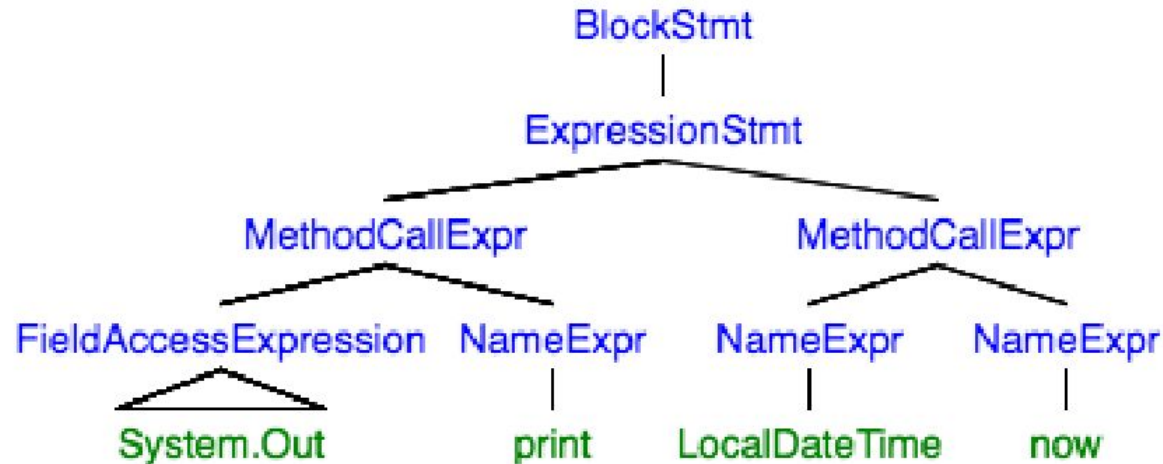
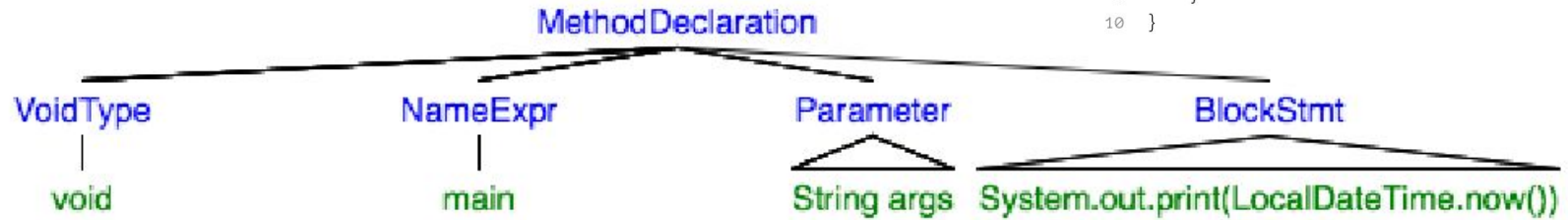
Java AST - node types

```
1  package com.github.javaparser;  
2  
3  import java.time.LocalDateTime;  
4  
5  public class TimePrinter {  
6  
7      public static void main(String args[]){  
8          System.out.print(LocalDateTime.now());  
9      }  
10 }
```



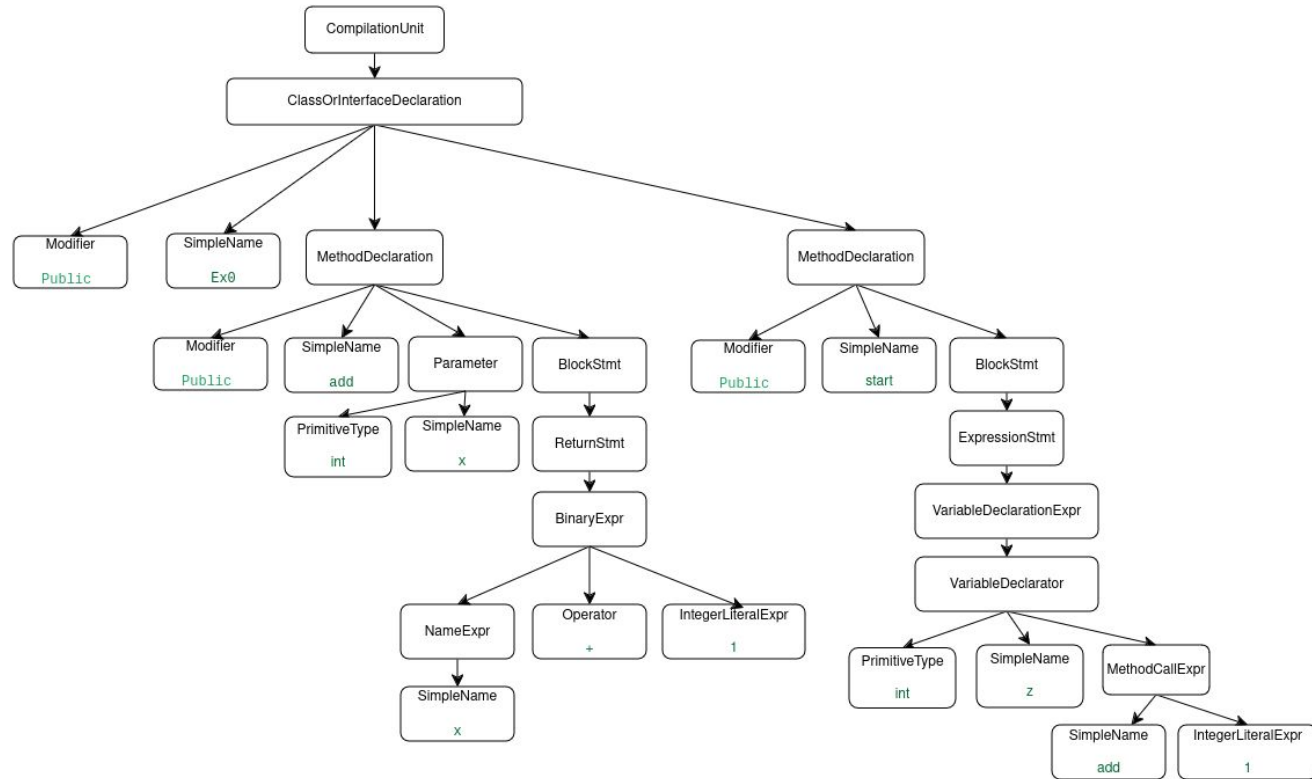
Java AST - node types

```
1 package com.github.javaparser;  
2  
3 import java.time.LocalDateTime;  
4  
5 public class TimePrinter {  
6  
7     public static void main(String args[]){  
8         System.out.print(LocalDateTime.now());  
9     }  
10 }
```



What would the AST look like?

```
public class Ex0 {  
  
    public int add(int x) {  
        return x + 1;  
    }  
  
    public void start() {  
        int z = add(1);  
    }  
}
```



Summary

- Java Bytecode Part II
 - Method Descriptors
- ASTs
 - Preserves hierarchical structure
 - Language dependent
- HW2 deadline extended
 - Will be released on Thursday
 - Due Friday September 25th